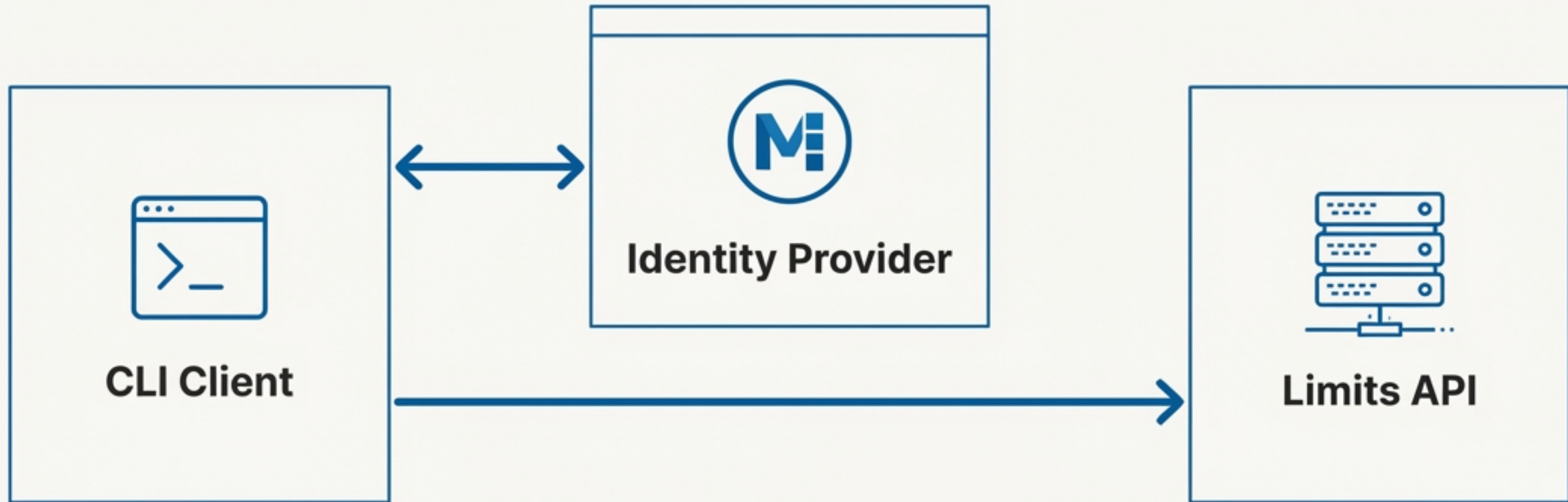


Building a Secure Application Ecosystem with Entra ID

A step-by-step guide from initial authentication to production-ready API security



The Core Challenge: Secure Login for a Public Client

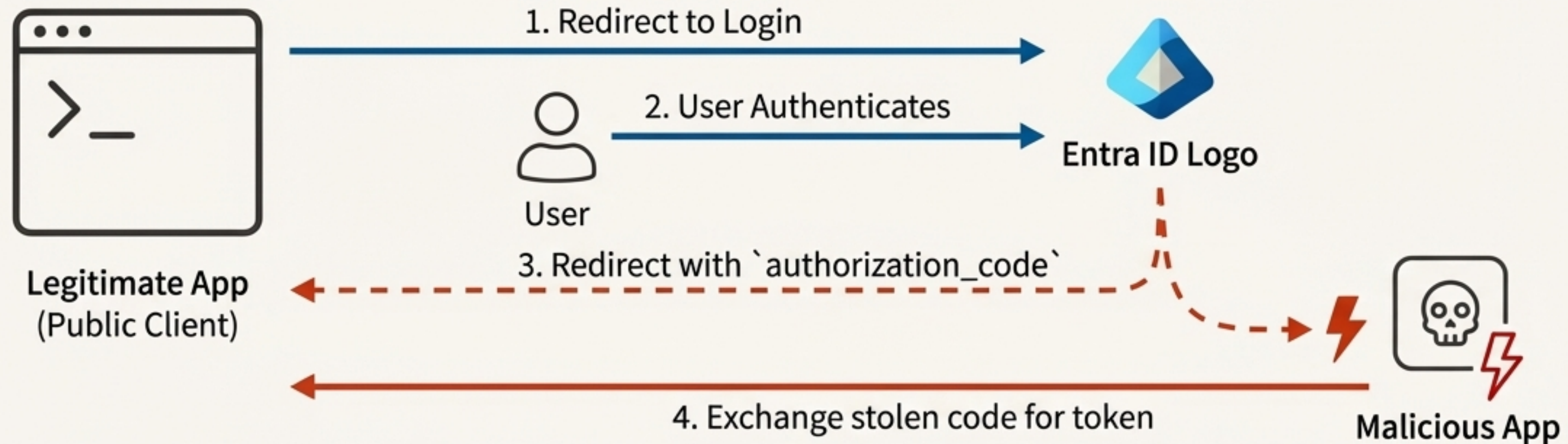
- Our goal is to build a Go-based command-line application.
- The application must authenticate a user to access protected resources.
- **Critical Constraint:** The application is a “public client.” It runs on a user’s machine and cannot be trusted to store a secret. It must never handle the user’s password directly.



The Standard Flow Contains a Hidden Vulnerability

The traditional OAuth 2.0 Authorization Code Flow is a common starting point. However, it was designed for confidential clients (like web servers) that can protect a secret. For public clients like desktop or CLI apps, this flow can be exploited.

The Authorization Code Interception Attack



The authorization code is like a single-use key. If it's stolen before our app can use it, an attacker gains access.

Securing the Handshake with PKCE

PKCE (Proof Key for Code Exchange, pronounced ‘**pixy**’) is an extension to the Authorization Code Flow that eliminates this vulnerability. It cryptographically **binds** the **initial** authorization request to the **final token exchange**, ensuring that even if the code is intercepted, it is **useless** to an attacker.

PKCE doesn't rely on a static, long-lived client secret. Instead, it uses a dynamic, one-time-use secret for each login request.

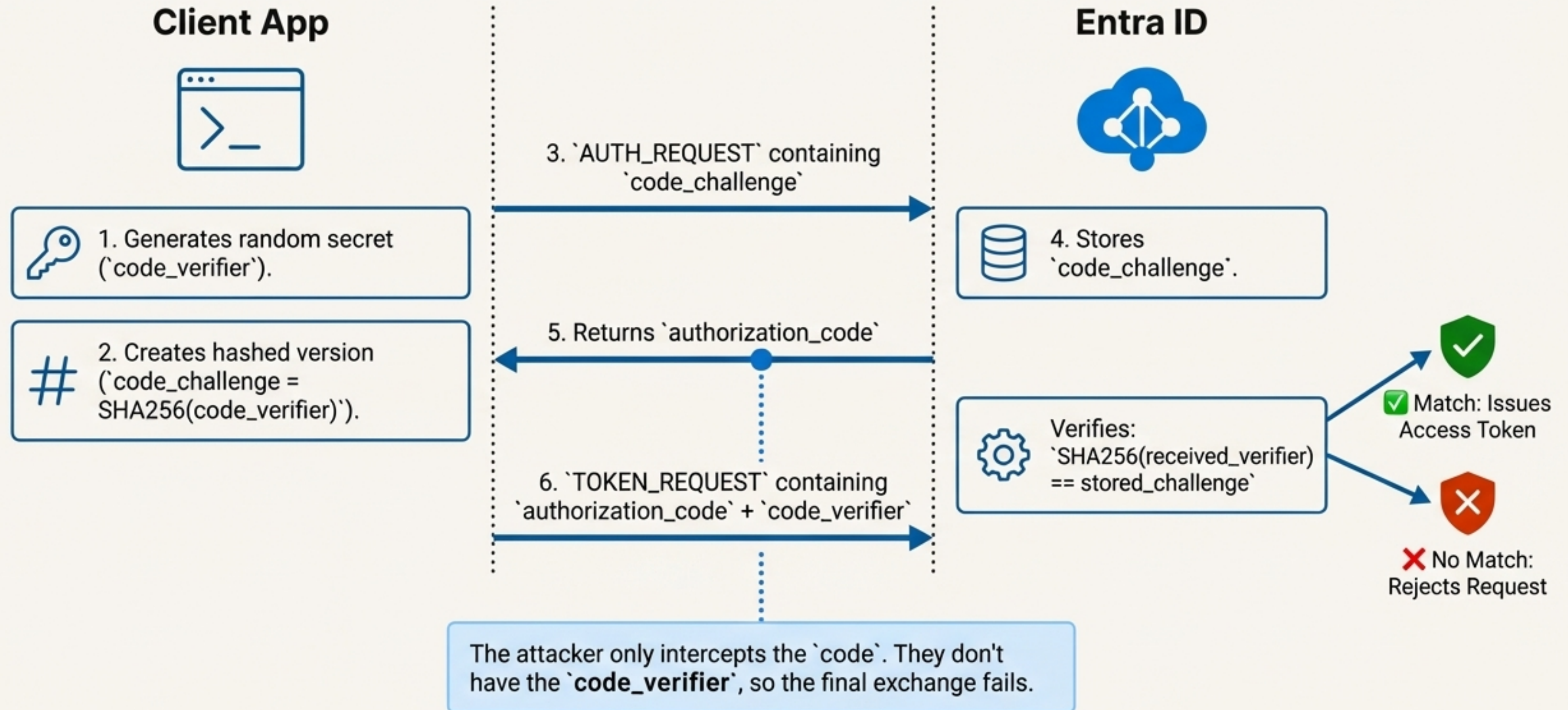


Hold a stolen
authorization_code



Microsoft Entra
ID Logo

How How PKCE Works: A Cryptographic Link



Bringing PKCE to the Command Line

The Code

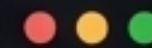
```
func generateCodeVerifier() string {  
    b := make([]byte, 32)  
    rand.Read(b)  
    return base64.RawURLEncoding.EncodeToString(b)  
}  
  
func generateCodeChallenge(verifier string) string {  
    hash := sha256.Sum256([]byte(verifier))  
    return base64.RawURLEncoding.EncodeToString(hash[:])  
}
```

Create a cryptographically random 32-byte secret.

Hash the secret using SHA-256 and Base64 encode it.

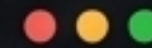
The User Experience

Good



```
Please open this URL manually:  
https://login.microsoftonline.com/common/oauth2/v2.  
0/authorize?client_id=...
```

Better

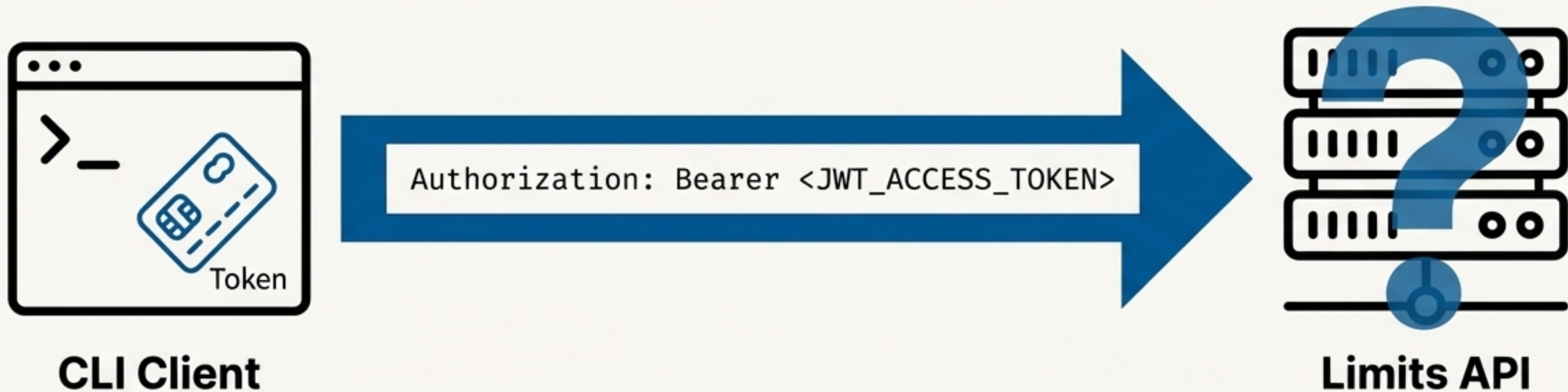


```
Opening browser for authentication...  
✓ Browser opened successfully.
```

We evolve the user experience from a manual copy-paste to an automatic browser launch, using ``os/exec`` to provide a seamless, cross-platform login.

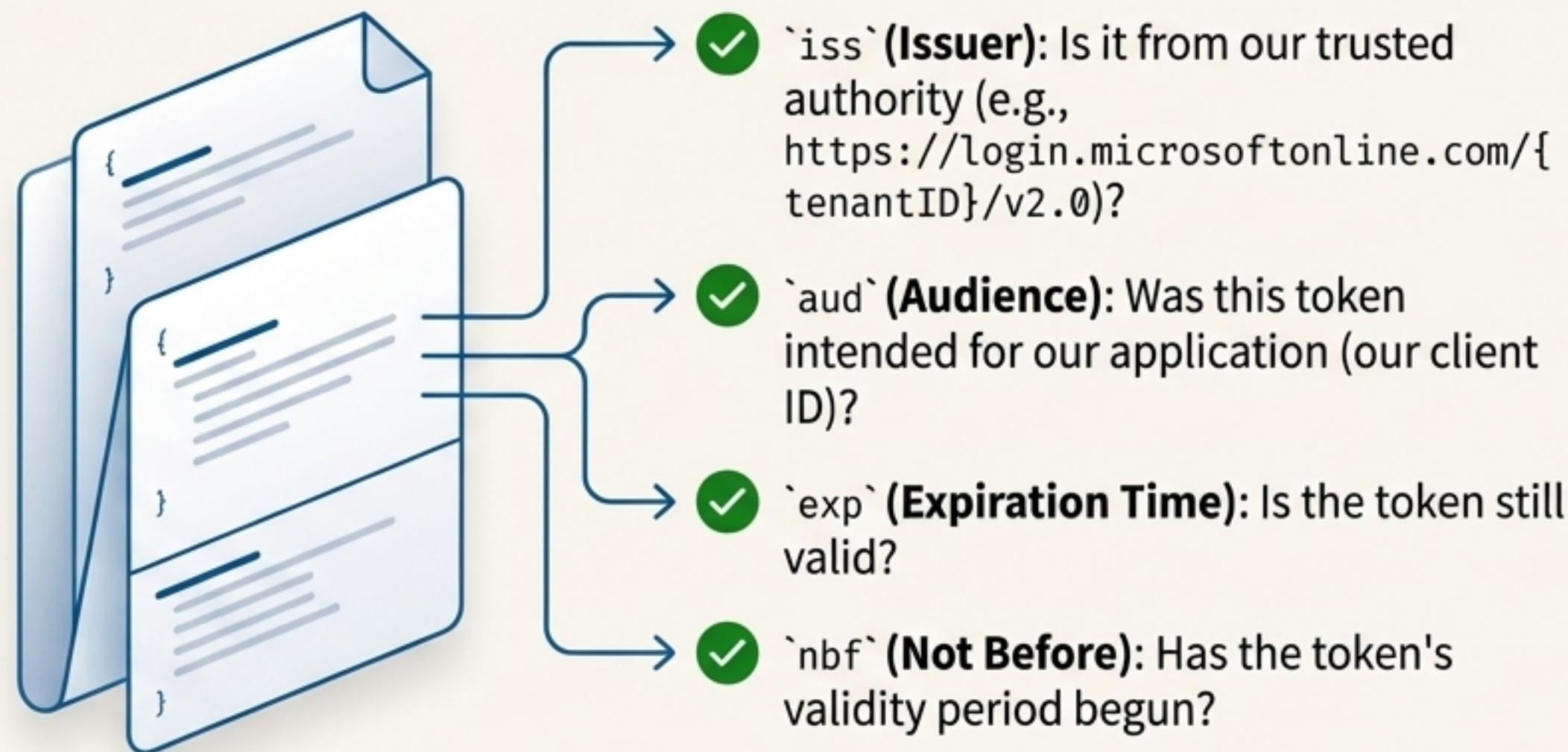
The Next Challenge: Securing the API Server

The client has successfully authenticated and received an access token. It now calls our `/limits` API, presenting this token as proof of identity. How does our API server ensure this token is valid, authentic, and trustworthy?



Initial Validation: Checking the Token's Claims

The first line of defense is to inspect the token's payload. A JSON Web Token (JWT) contains claims that provide essential information. We must validate these standard claims:

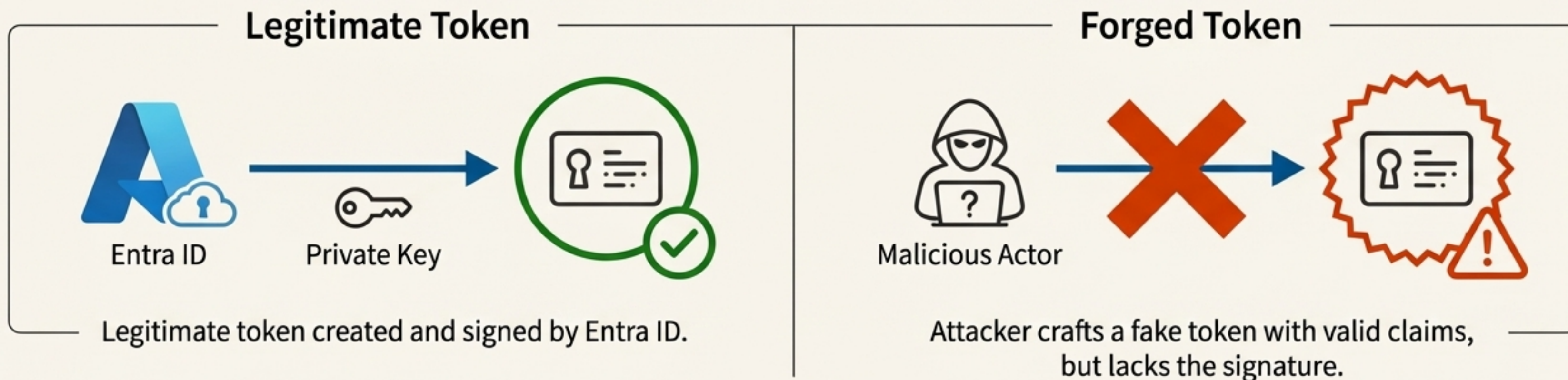


This is like checking the name, photo, and expiration date on a driver's license. It's crucial, but it doesn't prove the license itself isn't a high-quality fake.

The Real Question: Can This Token Be Forged?

Simply checking claims is not enough. Since the structure of a JWT is a public standard, an attacker could craft a fake token with perfectly valid-looking claims. Without a way to verify the token's authenticity, our API would accept it.

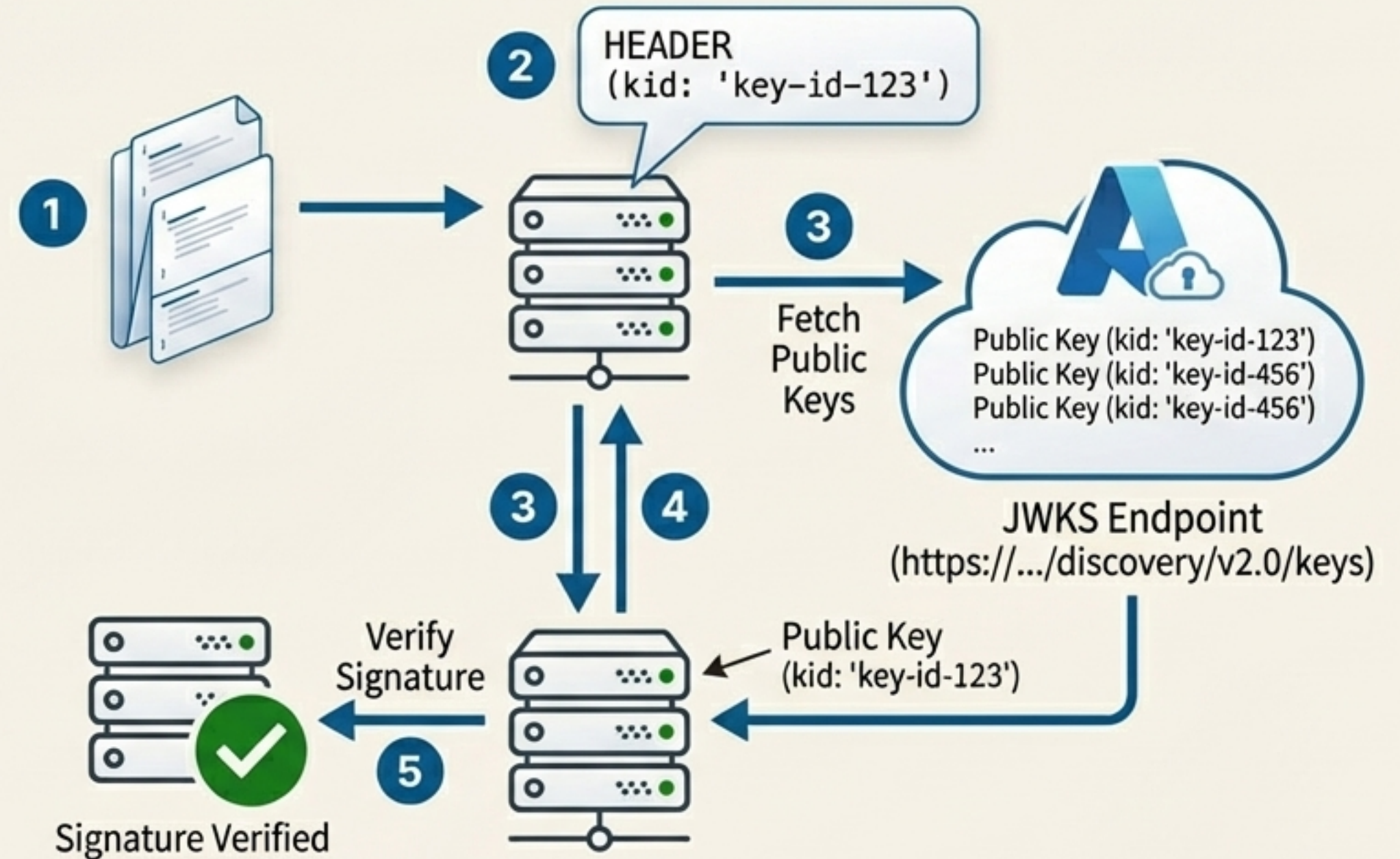
How do we prove that this token was actually created and signed by Entra ID and hasn't been tampered with?



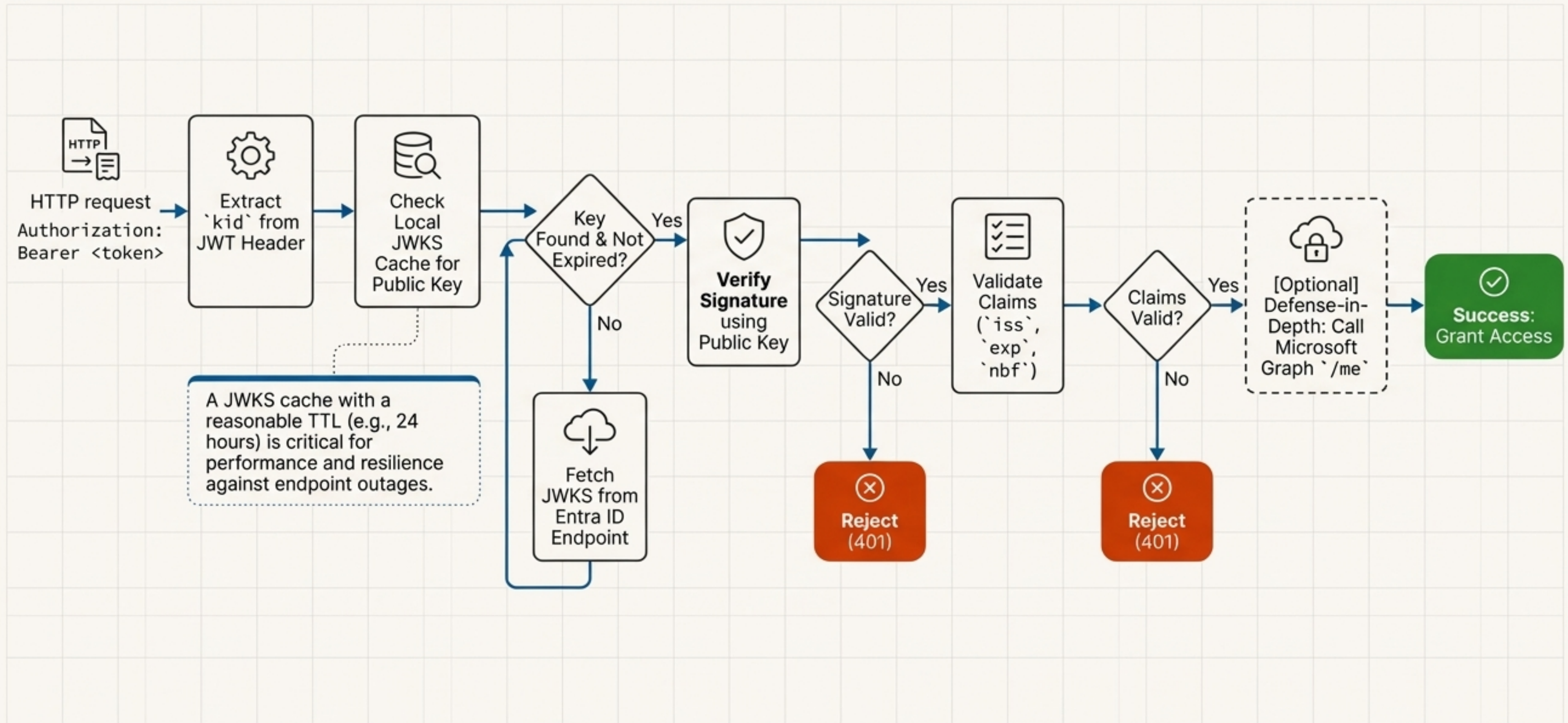
The Solution: Verifying Signatures with JWKS

Entra ID signs every JWT with a private key. We can verify this signature using a corresponding public key. Entra ID publishes these public keys at a well-known discovery endpoint called a JSON Web Key Set (JWKS).

Our server fetches the set of public keys from this endpoint. It uses the key specified in the token's header to verify the signature. This provides cryptographic proof that the token is authentic and was issued by Entra ID.

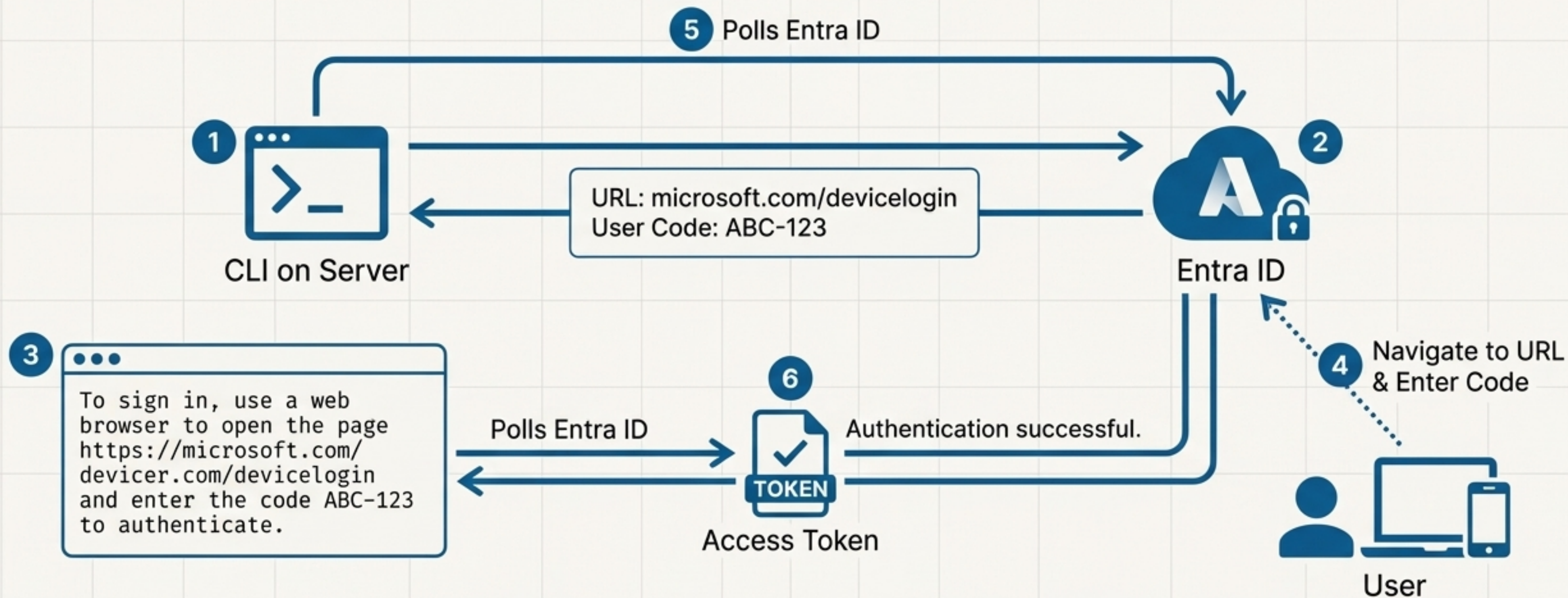


Production-Grade Token Validation: The Full Flow



Advanced Scenario: What If There Is No Browser?

The browser-based flow is ideal for user-attended desktops, but what about headless environments like an SSH session or a CI/CD pipeline? For this, OAuth 2.0 provides the Device Code Flow. It decouples the device requesting the token from the device performing the authentication.



Used by major CLI tools like Azure CLI, GitHub CLI, and `kubectl`.

A Security Checklist for Modern Authentication

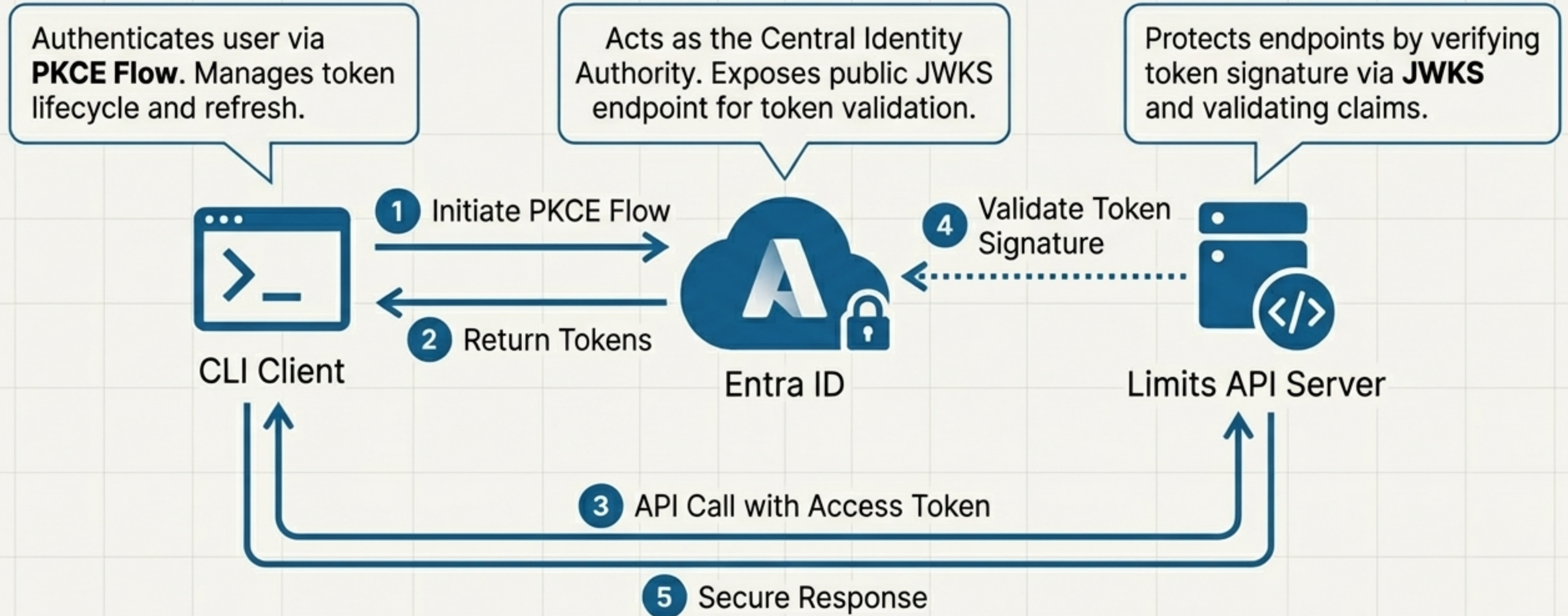
Client-Side (Public Clients)

- ✓ **ALWAYS** use the Authorization Code Flow with PKCE. It is the modern standard for desktop, mobile, and CLI apps.
- ✓ Use the `state` parameter in your authorization request to prevent Cross-Site Request Forgery (CSRF) attacks.
- ✓ Securely store tokens on the user's machine (e.g., using the OS keychain).

Server-Side (APIs)

- ✓ **ALWAYS** validate the token signature using the identity provider's public keys from the JWKS endpoint.
- ✓ **ALWAYS** validate the standard claims: issuer (iss), audience (aud), and expiration (exp).
- ✓ Enforce HTTPS (TLS) for all communication to prevent token interception over the network.

The Complete Secure Ecosystem



By combining a PKCE-secured client with a JWKS-protected server, we build a robust and trustworthy system on the foundation of modern identity standards.